

halfedge：共形映射模块设计文档

src/conformal.rs

halfedge 库

2026-07-05

目录

1	模块定位	1
2	调和映射：Dirichlet 能量最小化	2
2.1	连续变分问题	2
2.2	离散化	2
2.3	Euler-Lagrange 方程	2
2.4	稀疏线性系统求解	3
3	Möbius 变换：复数分式线性变换	3
3.1	一般公式	3
3.2	单位圆盘自同构：Blaschke 因子	3
3.3	复数算术细节	3
3.4	应用：圆盘参数化中心调整	3
4	共形比例因子：离散 Yamabe 方程	4
4.1	方程	4
4.2	零空间消除	4
4.3	求解与返回	4
5	算法流程图	5
6	退化守卫	5
7	使用示例	5
8	测试覆盖	6

1 模块定位

`conformal.rs` 在 `geometry.rs` 的余切拉普拉斯与曲率算子之上，叠加**共形映射**（Conformal Mapping）能力。共形映射保持角度（局部相似变换），在纹理映射、曲面配准、形状分析中有广泛应用。本模块提供 4 个公共函数，覆盖调和映射、Möbius 变换与离散 Yamabe 方程三类算子：

类别	API	语义
调和映射	<code>harmonic_map(mesh, correspondences)</code>	Dirichlet 能量最小映射，返回每顶点 3D 位置
Möbius 变换	<code>apply_mobius_transform(uv, a, b, c, d)</code>	复数分式线性变换 $f(z) = \frac{az+b}{cz+d}$ 逐点施加
	<code>mobius_to_center(target)</code>	返回将圆盘中心移到 <code>target</code> 的 Blaschke 系数
共形比例因子	<code>compute_vertex_scale_factors(mesh, K')</code>	解 $\Delta u = K - K'$ ，返回对数比例因子 u_i

- `harmonic_map` 与 `compute_vertex_scale_factors` 返回 `Option<T>`：无对应点 / 求解失败时返回 `None`，而非 `panic`；
- `apply_mobius_transform` 与 `mobius_to_center` 是纯函数，对任意输入均有定义（除零时复数除法回退为 0）；
- 复数算术以 `[f64; 2]` 表示 $z = x + iy$ ，运算细节见 §3。

2 调和映射：Dirichlet 能量最小化

2.1 连续变分问题

设源曲面 $\mathcal{M}$ ，目标表面上的稀疏对应 $\mathcal{C} = \{(v_k, \vec{g}_k)\}$ ，求解映射 $f : \mathcal{M} \rightarrow \mathbb{R}^3$ 使得 Dirichlet 能量最小：

$$E(f) = \frac{1}{2} \int_{\mathcal{M}} \|\nabla f\|^2 dA, \quad f(v_k) = \vec{g}_k \text{ (约束)}.$$

2.2 离散化

在三角网格上，Dirichlet 能量离散为顶点位置差的加权和：

$$E = \sum_{(i,j) \in \mathcal{E}} w_{ij} \|f_i - f_j\|^2, \quad w_{ij} = \frac{1}{2} (\cot \alpha_{ij} + \cot \beta_{ij}),$$

其中 α_{ij}, β_{ij} 是边 ij 两侧对角， w_{ij} 即 `geometry::cotan_edge_weight` 的 1/2 倍。

2.3 Euler-Lagrange 方程

对自由顶点 i ，最小化 E 给出余切拉普拉斯方程：

$$\sum_{j \in N(i)} w_{ij} (f_i - f_j) = 0 \iff (Lf)_i = 0,$$

对应顶点 i 处约束 $f_i = \vec{g}_i$ 。整体形式为

$$L f = b, \quad b_i = \begin{cases} \vec{g}_i & i \in \mathcal{C} \\ 0 & \text{otherwise} \end{cases}$$

固定行替换为单位行，对角正则化 $\varepsilon = 10^{-8}$ 保证数值稳定。

2.4 稀疏线性系统求解

矩阵 L 由 `build_full_cotan_laplacian` 构造为 `SparseSystem`，三坐标 (x, y, z) 独立求解：

$$L f_x = b_x, \quad L f_y = b_y, \quad L f_z = b_z.$$

使用共轭梯度法 `conjugate_gradient`，容差 10^{-6} ，最大迭代 $100n$ 。任一分量求解失败则整体返回 `None`。

3 Möbius 变换：复数分式线性变换

3.1 一般公式

Möbius 变换是复平面上的分式线性映射：

$$f(z) = \frac{az + b}{cz + d}, \quad ad - bc \neq 0.$$

在单位圆盘 $\mathbb{D} = \{z : |z| < 1\}$ 上，Möbius 变换构成 $\mathbb{D} \rightarrow \mathbb{D}$ 的共形自同构群。

3.2 单位圆盘自同构：Blaschke 因子

将圆心 0 映到 a ($|a| < 1$) 的标准 Blaschke 因子为

$$f(z) = \frac{z - a}{1 - \bar{a}z},$$

对应 Möbius 系数

$$a_{\text{mob}} = 1, \quad b_{\text{mob}} = -a, \quad c_{\text{mob}} = -\bar{a}, \quad d_{\text{mob}} = 1.$$

关键性质： $f(0) = -a$ ， $f(a) = 0$ ， $|f(z)| = 1$ 当 $|z| = 1$ 。`mobius_to_center(target)` 取 $a = \text{target}$ 返回上述系数。

3.3 复数算术细节

以 $z = x + iy$ 表示复数（[f64; 2] 即 $[x, y]$ ）。复数乘除公式：

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i, \quad (1)$$

$$\frac{a + bi}{c + di} = \frac{(ac + bd) + (bc - ad)i}{c^2 + d^2}. \quad (2)$$

实现细节：

- `complex_mul(a, b)` 返回 $[a_0b_0 - a_1b_1, a_0b_1 + a_1b_0]$ ；
- `complex_mul_add(a, b, c)` 计算 $a \cdot b + c$ ；
- `complex_div(a, b)` 计算 a/b ，当分母 $c^2 + d^2 < 10^{-14}$ 时返回 $[0, 0]$ （除零守卫）。

3.4 应用：圆盘参数化中心调整

给定圆盘参数化 UV 数组，施加 `mobius_to_center` 后再 `apply_mobius_transform`，可将原参数化的中心 0 移到任意 $|a| < 1$ 目标点，同时保持边界仍在单位圆上——这是共形参数化边界重分布的标准工具。

4 共形比例因子：离散 Yamabe 方程

4.1 方程

离散 Yamabe 方程将当前高斯曲率 K 调整到目标 K' ：

$$\Delta u = K - K',$$

其中 Δ 是余切拉普拉斯， u_i 是顶点 i 的对数共形比例因子。度规变换 $\ell'_{ij} = \ell_{ij} \cdot e^{(u_i+u_j)/2}$ 给出共形新度量。

4.2 零空间消除

余切拉普拉斯有常数零空间 ($\Delta \mathbf{1} = 0$)，方程不唯一。实现采用 **pin 顶点 0** 策略：

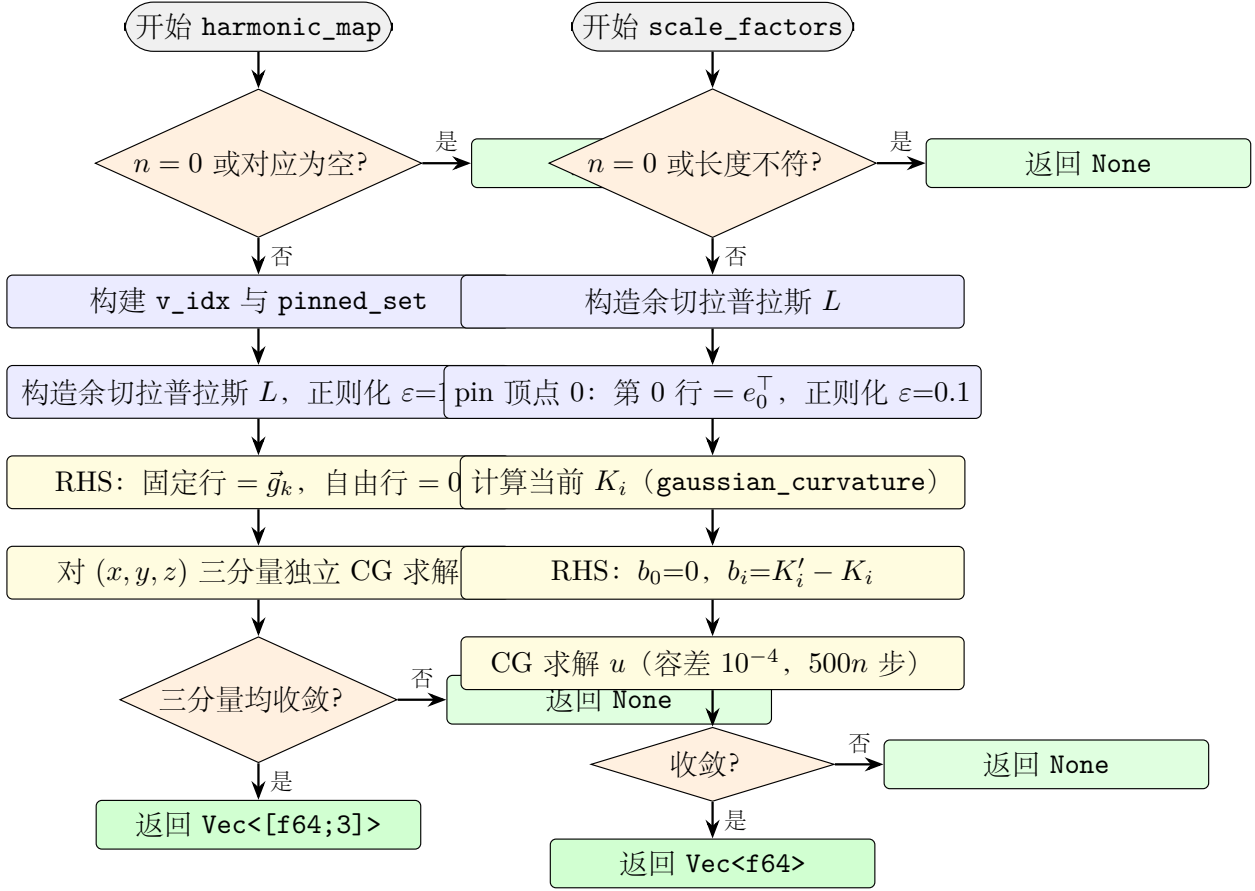
- 矩阵第 0 行替换为单位行 $e_0^\top$ (`sys.add_diag(0, 1.0)`)；
- 其余行保留原拉普拉斯系数；
- RHS 第 0 行设为 0 ($u_0 = 0$)，其余行 $b_i = K'_i - K_i$ 。

对角正则化系数 $\varepsilon = 0.1$ 进一步保证 CG 收敛。

4.3 求解与返回

`conjugate_gradient` 容差 10^{-4} ，最大迭代 $500n$ 。 u_i 用于 Circle Pattern / 离散 Ricci 流的后续步骤： e^{u_i} 即顶点 i 处的共形缩放因子。

5 算法流程图



6 退化守卫

函数	退化场景	处理
harmonic_map	无对应点 (correspondences 为空)	返回 None
harmonic_map	CG 任一分量未收敛	返回 None
harmonic_map	空网格 ($n = 0$)	返回 None
apply_mobius_transform	分母 $c^2 + d^2 < 10^{-14}$ ($cz + d \approx 0$)	该点返回 $[0, 0]$
mobius_to_center	$ a \geq 1$ (target 超出单位圆)	系数仍生成, 但映射不稳定
compute_vertex_scale_factors	target_curvature 长度 $\neq n$	返回 None
compute_vertex_scale_factors	空网格 ($n = 0$)	返回 None
compute_vertex_scale_factors	CG 未收敛	返回 None

7 使用示例

```

1 use halfedge::conformal::{
2   harmonic_map, apply_mobius_transform, mobius_to_center,

```

```

3      compute_vertex_scale_factors,
4  };
5  use halfedge::test_util::build_icosphere;
6
7  // 1)          icosphere
8  let mesh = build_icosphere(2);
9  let vids: Vec<_> = mesh.vertex_ids().collect();
10 let correspondences = vec![
11     (vids[0], [0.0, 0.0, 1.0]),
12     (vids[1], [1.0, 0.0, 0.0]),
13 ];
14 if let Some(mapped) = harmonic_map(&mesh, &correspondences) {
15     assert_eq!(mapped.len(), mesh.vertex_count());
16 }
17
18 // 2) Mobius           $f(z) = (1*z + 0) / (0*z + 1)$ 
19 let uv = vec![[0.5, 0.3], [-0.2, 0.8]];
20 let id_uv = apply_mobius_transform(
21     &uv, [1.0, 0.0], [0.0, 0.0], [0.0, 0.0], [1.0, 0.0],
22 );
23 assert!((id_uv[0][0] - 0.5).abs() < 1e-12);
24
25 // 3)          (0.5, 0.3)
26 let (a, b, c, d) = mobius_to_center([0.5, 0.3]);
27 let moved = apply_mobius_transform(&[[0.0, 0.0]], a, b, c, d);
28 //  $f(0) = -target$ 
29 assert!((moved[0][0] + 0.5).abs() < 1e-12);
30
31 // 4)          0
32 let n = mesh.vertex_count();
33 let target_k = vec![0.0; n];
34 if let Some(u) = compute_vertex_scale_factors(&mesh, &target_k) {
35     assert_eq!(u.len(), n);
36     assert!(u.iter().all(|x| x.is_finite()));
37 }

```

8 测试覆盖

测试名	验证点
test_mobius_identity	恒等系数 (1, 0, 0, 1) 不改变输入 UV
test_mobius_translation	$f(z) = z + 1$ 平移变换正确
test_mobius_to_center_maps_origin	Blaschke 因子满足 $f(0) = -target$
test_scale_factors_sphere	球面零目标曲率下 u_i 全有限且长度正确

src/conformal.rs 共 4 个单元测试，全库共 371 个。